

UNINTER

Projeto Multidisciplinar - Trilha Back-end

Rede Raízes do Nordeste

Aluno: _____

RU: _____

Curso: Análise e Desenvolvimento de Sistemas

Ano: 2026

Sumário

1. Introdução e objetivos
2. Análise e requisitos
3. Modelagem e diagramas
4. Arquitetura
5. API e endpoints
6. LGPD, segurança e auditoria
7. Entrega técnica
8. Plano de testes
9. Conclusão
10. Referências

1. Introdução e objetivos

Este trabalho apresenta uma proposta de back-end para a rede de lanchonetes Raízes do Nordeste. A solução foi pensada para atender uma operação com múltiplas unidades e múltiplos canais de venda, como aplicativo, totem, balcão, pickup e web. O foco do projeto está na construção de uma API REST com persistência em banco, autenticação, controle de perfis, estoque por unidade, pedidos, pagamento simulado, fidelidade e logs de auditoria.

O objetivo técnico principal é demonstrar um fluxo completo e testável: o cliente realiza um pedido informando o canal de origem, o sistema valida os itens e o estoque da unidade, registra o pedido, simula o pagamento por meio de um gateway mock e permite a atualização de status pela cozinha ou gerência.

2. Análise e requisitos

2.1 Requisitos funcionais

- RF01 - Permitir cadastro e autenticação de usuários com perfis CLIENTE, ATENDENTE, COZINHA, GERENTE e ADMIN.
- RF02 - Permitir consulta de unidades da rede e cardápio por unidade.
- RF03 - Permitir cadastro e manutenção de produtos e itens de cardápio por unidade.
- RF04 - Permitir consulta e movimentação de estoque por unidade.
- RF05 - Permitir criação de pedido com itens, total, status e canalPedido obrigatório.
- RF06 - Permitir consulta e filtro de pedidos por canalPedido e status.
- RF07 - Permitir pagamento mock com retorno aprovado ou recusado.
- RF08 - Permitir atualização do status do pedido pela cozinha ou gerência.
- RF09 - Permitir programa de fidelidade simples, com crédito de pontos mediante consentimento.
- RF10 - Registrar logs de ações sensíveis, como criação de pedido, movimentação de estoque, pagamento e alteração de status.

2.2 Requisitos não funcionais

- RNF01 - A API deve usar autenticação por token JWT.
- RNF02 - Senhas devem ser armazenadas com hash seguro.
- RNF03 - Endpoints sensíveis devem aplicar autorização por perfil.
- RNF04 - A API deve documentar seus contratos por Swagger/OpenAPI.
- RNF05 - As respostas de erro devem seguir um padrão JSON único.
- RNF06 - O sistema deve possuir README reproduzível, seed e coleção Postman.
- RNF07 - A integração de pagamento deve ser tolerante a falhas, sem depender de provedor real.
- RNF08 - Dados pessoais devem seguir princípios de minimização e finalidade da LGPD.

2.3 Priorização do MVP

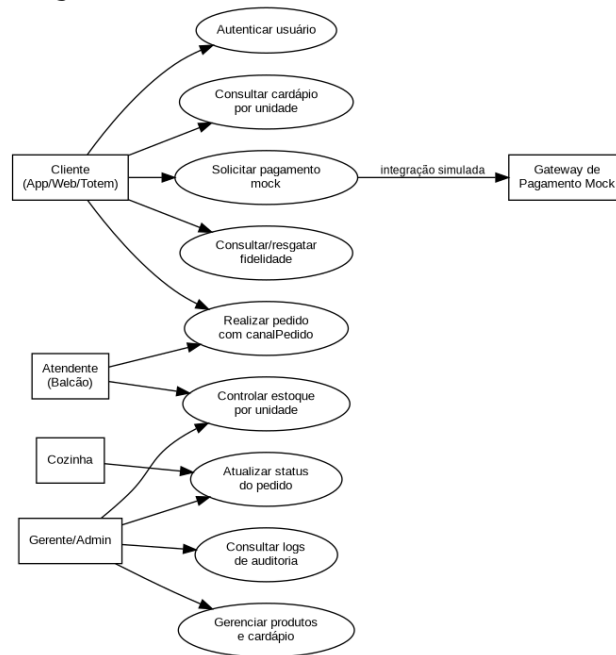
O MVP priorizado foi o fluxo Pedido -> Pagamento mock -> Atualização de status, pois ele conecta os principais pontos do estudo de caso: multicanalidade, estoque, pedido, integração simulada, autenticação, autorização, fidelidade e auditoria. Funcionalidades de promoções foram documentadas como evolução possível, pois dependem de regras comerciais mais variáveis.

3. Modelagem e diagramas

A modelagem foi construída para manter coerência entre domínio, banco de dados e endpoints. O pedido é a entidade central do fluxo crítico e possui relacionamento com cliente, unidade, itens, pagamento e logs.

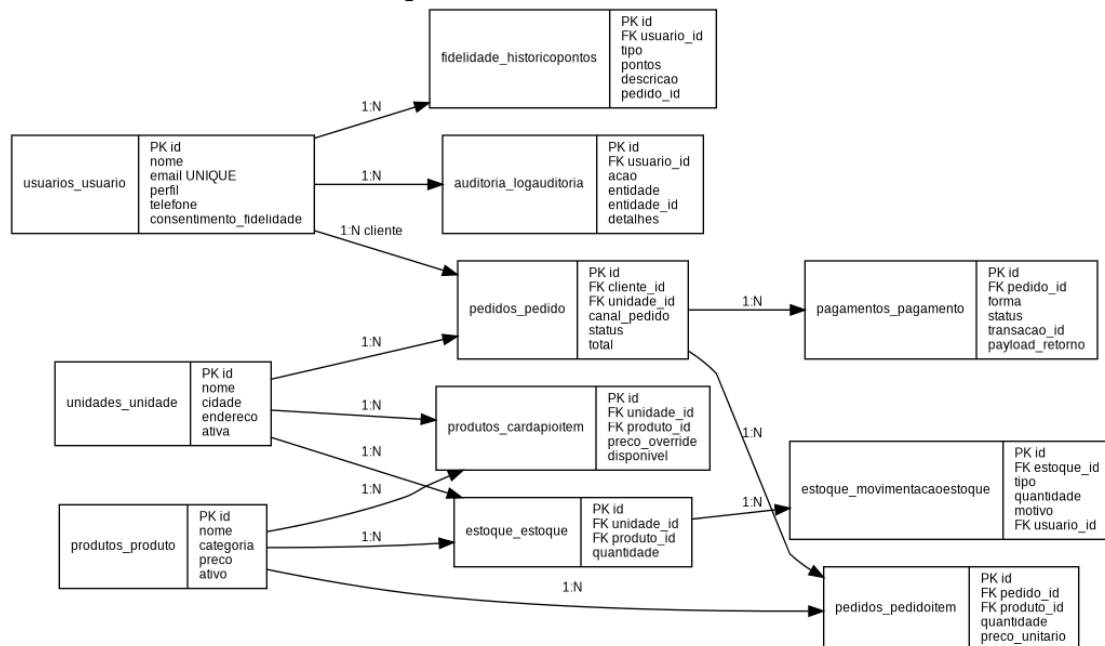
3.1 Diagrama de casos de uso

Diagrama de Casos de Uso - Raízes do Nordeste



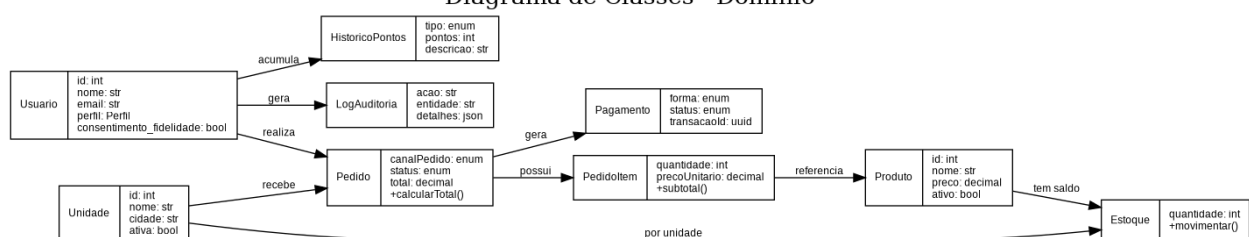
3.2 DER simplificado

DER simplificado - Raízes do Nordeste



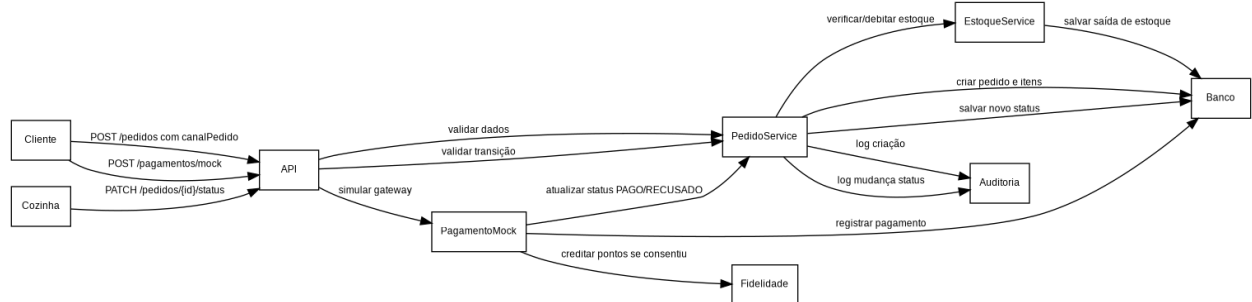
3.3 Diagrama de classes

Diagrama de Classes - Domínio



3.4 Sequência do fluxo crítico

Sequência - Pedido, pagamento mock e status



4. Arquitetura

O projeto foi organizado por apps do Django, separando responsabilidades por domínio funcional. A estrutura segue uma abordagem em camadas simples: API/Controllers nas views, Application nos services, Domain nos models e Infrastructure no ORM, seed, documentação e integração mock.

Camada	Responsabilidade	Arquivos principais
API	Receber requisições, aplicar permissões e serializar respostas.	views.py, serializers.py, urls.py
Application	Orquestrar fluxos e regras de negócio.	services.py
Domain	Representar entidades, enums e relações.	models.py
Infrastructure	Persistência, seed, Swagger, mock e auditoria.	ORM, management commands, drf-spectacular

5. API e endpoints

Os endpoints foram organizados por recurso. As URLs usam plural, IDs no path e filtros por query params. O campo canalPedido é obrigatório na criação do pedido e pode ser usado em consultas.

Recurso	Método/Rota	Permissão	Finalidade
Auth	POST /api/auth/login/	Público	Autenticar e retornar JWT
Usuários	GET /api/usuarios/me/	JWT	Consultar usuário autenticado
Unidades	GET /api/unidades/	JWT	Listar unidades
Produtos	GET/POST /api/produtos/	JWT / Gerente	Consultar ou gerenciar produtos
Cardápio	GET /api/cardapio/?unidade_id=1	JWT	Consultar cardápio por unidade
Estoque	POST /api/estoques/movimentar/	Gerente/Admin	Registrar entrada, saída ou ajuste
Pedidos	POST /api/pedidos/	JWT	Criar pedido com canalPedido
Pedidos	GET /api/pedidos/?canalPedido=TOTEM	JWT	Filtrar pedidos por canal
Pedidos	PATCH /api/pedidos/{id}/status/	Cozinha/Gerente/Admin	Atualizar status
Pagamentos	POST /api/pagamentos/mock/	JWT	Simular pagamento
Fidelidade	GET /api/fidelidade/saldo/	JWT	Consultar saldo de pontos
Auditoria	GET /api/auditoria/logs/	Gerente/Admin	Consultar logs

5.1 Exemplo de request - criação de pedido

```
{
  "unidadeId": 1,
  "canalPedido": "TOTEM",
  "itens": [ { "produtoId": 1, "quantidade": 2 } ],
  "formaPagamento": "MOCK"
}
```

5.2 Padrão de erro

```
{
  "error": "ESTOQUE_INSUFICIENTE",
  "message": "Não há quantidade suficiente para um ou mais itens.",
  "details": [ { "field": "produtoId", "issue": "Saldo insuficiente" } ],
  "timestamp": "2026-02-05T12:00:00Z",
  "path": "/api/pedidos/"
}
```

6. LGPD, segurança e auditoria

A solução aplica controles mínimos de segurança e privacidade no back-end. O login usa JWT, as senhas são armazenadas com hash pelo Django e os endpoints administrativos possuem autorização por perfil. A API não retorna senha ou hash em respostas.

Controle	Aplicação no projeto
Hash de senha	Feito automaticamente pelo Django User/AbstractUser.
JWT	Login em /api/auth/login/ e uso de Authorization: Bearer token.
Roles	Perfis CLIENTE, ATENDENTE, COZINHA, GERENTE e ADMIN.
LGPD	Coleta mínima: nome, e-mail, telefone opcional e consentimento de fidelidade.
Fidelidade	Pontos só são creditados quando consentimento_fidelidade é verdadeiro.
Auditoria	Registra criação de pedido, pagamento, estoque, status e fidelidade.
Pagamento	Mock sem dados reais de cartão ou provedor externo.

7. Entrega técnica

O repositório contém o código-fonte da API, README de execução, .env.example, documentação técnica, diagramas e coleção Postman. Para correção, o avaliador deve clonar o repositório, instalar dependências, executar migrations, rodar o seed e iniciar o servidor local.

A documentação Swagger fica disponível em /api/docs/ após iniciar a aplicação. A coleção Postman está em docs/postman/raizes_do_nordeste.postman_collection.json.

8. Plano de testes

Foram definidos 12 cenários de teste, com 6 ou mais positivos e pelo menos 4 negativos, cobrindo autenticação, autorização, validação de dados, regra de estoque, pagamento mock e auditoria.

ID	Cenário	Endpoint	Esperado
T01	Login válido	POST /api/auth/login/	200 + access token
T02	Login inválido	POST /api/auth/login/	401 + erro padrão
T03	Listar cardápio	GET /api/cardapio/	200 + itens
T04	Criar pedido válido	POST /api/pedidos/	201 + AGUARDANDO_PAGAMENTO
T05	Pedido sem token	POST /api/pedidos/	401
T06	Canal inválido	POST /api/pedidos/	400/422
T07	Produto inexistente	POST /api/pedidos/	404
T08	Estoque insuficiente	POST /api/pedidos/	409
T09	Pagamento aprovado	POST /api/pagamentos/mock/	201 + APROVADO
T10	Pagamento recusado	POST /api/pagamentos/mock/	201 + RECUSADO
T11	Cliente atualiza status	PATCH /api/pedidos/{id}/status/	403
T12	Listar auditoria	GET /api/auditoria/logs/	200 + logs

9. Conclusão

O projeto implementa um back-end funcional e reproduzível para a rede Raízes do Nordeste, atendendo ao fluxo crítico escolhido. A solução conecta os diagramas e o modelo de dados aos endpoints implementados, com persistência real, autenticação JWT, permissões por perfil, campo canalPedido, pagamento mock, fidelidade com consentimento e logs de auditoria.

Como evolução, o sistema poderia receber regras mais completas de promoções, deploy em nuvem, testes automatizados mais abrangentes e uma rotina formal de anonimização de dados antigos. Mesmo assim, o MVP entregue cobre os principais requisitos funcionais e não funcionais previstos para a trilha Back-end.

10. Referências

- Django Software Foundation. Documentação oficial do Django.
- Django REST Framework. Documentação oficial do DRF.
- drf-spectacular. Documentação de geração OpenAPI/Swagger.
- Brasil. Lei Geral de Proteção de Dados Pessoais - LGPD, Lei nº 13.709/2018.
- Roteiro da Atividade Prática - Projeto Multidisciplinar - Trilha Back-end, UNINTER, 2026.